

Tool Survey Proposal - Web Automation Testing

Group 06

June 01, 2026

Project Overview

Course: CSC15003 - Software Testing

Topic: T02 - Web Automation Testing

Software Under Test (SUT): [EShop](#) (React + Node.js/Express)

Team Members:

- 23127378 - Nguyễn Gia Huy
- 23127379 - Thái Minh Huy
- 23127396 - Lương Linh Khôi
- 23127449 - Nguyễn Tấn Phát

Table of Contents

- [1. Theoretical Foundation of Web Automation Testing](#)
- [2. Scope of Testing \(End-to-End User Journeys\)](#)
- [3. Comprehensive Tool Assessment](#)
- [4. Comprehensive Comparison Matrix](#)
- [5. Final Recommendation & Rationale](#)
- [6. AI Disclosure & Cross-Verification](#)

1. Theoretical Foundation of Web Automation Testing

1.1. Core Concept & Business Value

Automation Testing is the systematic design, implementation, and execution of test cases through software-driven validation rather than manual interaction. Its primary business value is executing **Regression Testing** at scale to protect critical workflows, reduce defect leakage, and compound long-term ROI by lowering the marginal cost of verification as the product evolves.

A fast feedback loop is essential for modern engineering because automated checks must surface failures shortly after code changes are introduced, enabling rapid diagnosis and safe iteration. This capability is a prerequisite for CI/CD pipelines, where continuous validation acts as a quality gate and prevents unstable changes from propagating downstream.

1.2. The Specifics of Web Automation

Web automation is uniquely challenging in modern React-based SPAs because the UI is driven by **Asynchronous DOM** rendering, client-side routing, and network latency that reshapes state on the fly. Elements appear and disappear with state transitions, route changes do not trigger full page reloads, and API response timing can shift the visible UI, so test reliability depends on tight synchronization with rendering and network conditions rather than static page structure.

Flakiness is the ultimate risk in web automation: tests fail intermittently despite unchanged code because async race conditions, non-deterministic rendering, or variable network timing produce unstable observations. Flaky results erode trust in the suite, slow delivery through false alarms, and force teams to re-validate outcomes manually.

1.3. Fundamental Principles

- **Locator Resilience:** Favor static, semantic attributes such as `data-testid` or stable IDs over brittle CSS chains or positional XPath to reduce breakage from UI refactors and re-rendered components.
- **Smart Synchronization:** Use explicit, state-based waits that align with rendering and network completion instead of hard sleeps, which inflate runtime and still fail under variable latency.
- **Test Isolation:** Ensure independent setup and teardown so tests do not share mutable state, improving repeatability and enabling safe parallel execution.

1.4. Web Automation Lifecycle

1. **Test Strategy:** Define risk-based scope, target coverage, and acceptance criteria.
2. **Scripting:** Implement test logic with reusable abstractions and consistent assertions.
3. **Execution:** Run suites locally and in CI with reporting and artifacts.
4. **Maintenance:** Update selectors and flows as the application evolves.

Modern execution relies heavily on headless CLI environments to maximize speed, minimize GUI overhead, and integrate cleanly into CI/CD workflows, aligning with terminal-native practices and standard CLI pipelines.

1.5. The Role of AI in Web Automation Testing

AI reduces historical web automation pain points by augmenting robustness and authoring speed. **Self-healing locators** use attribute-weight models to adapt to DOM changes and prioritize stable signals, reducing failures caused by brittle selectors and improving resilience against UI refactors.

AI-assisted script generation applies LLMs to draft and refactor test logic from high-level intent, accelerating coverage growth while keeping structure consistent. Together, these capabilities reduce the heavy maintenance burden typical of UI testing by shifting effort from constant rewrites toward higher-level review and validation.

2. Scope of Testing (End-to-End User Journeys)

Journey 1: The Core Purchasing Lifecycle (Happy Path)

Objective: Validate the complete customer purchase flow from product discovery through successful checkout with a valid discount.

1. Search for a product by keyword and confirm the product list updates correctly to locate a target item `[FR-05]`.
2. Open product details, verify required information and quantity constraints, then add the item to the cart `[FR-06]`.
3. In the cart, adjust quantity, verify line totals and the "Tong cong" label, then proceed to checkout `[FR-07]`.

4. Attempt checkout while unauthenticated to confirm login is required, then authenticate with a valid user account [FR-08] [FR-02].
5. Apply a coupon that satisfies all five conditions and verify discount calculation and final amount at checkout [FR-09] [FR-08].
6. Submit the order successfully and confirm the cart is cleared after completion [FR-08].

Journey 2: Security Boundaries & Account Recovery (Negative Path)

Objective: Validate account logout, password recovery, and profile updates under security constraints.

1. Perform three consecutive failed logins to trigger the 30-second logout and verify the error response does not over-disclose details [FR-02].
2. Start the two-step Forgot Password process, request the OTP for a registered email, and verify the step indicator is shown [FR-03].
3. Reset the password with a compliant strong password and matching confirmation, then verify rejection for invalid or mismatched input [FR-03] [FR-01].
4. Log in with the new password and update profile fields while confirming email remains immutable and role cannot be altered [FR-02] [FR-04].

Journey 3: The Admin Order State Machine (Admin Path)

Objective: Validate admin access controls and enforce the order state transitions for a live customer order.

1. Log into the Admin portal with an admin account and confirm access is gated by role and token [FR-12].
2. Optionally import products via CSV and verify atomic rollback if any row fails validation [FR-16].
3. Locate the order created in Journey 1 and transition it from pending to confirmed [FR-10] [FR-18].
4. Transition the order from confirmed to shipping and then to delivered, verifying each state change is accepted [FR-10] [FR-18].
5. Attempt to cancel a delivered order and confirm the action is rejected because delivered is a final state [FR-10] [FR-18].

Journey 4: Admin Catalog and Coupon Lifecycle to Customer Verification

Objective: Validate that admin-managed catalog and coupon data propagate end-to-end to customer purchase flows.

1. As admin, create a category and a product with required fields and a positive price, then verify it appears in the storefront [FR-14] [FR-15].
2. As admin, create a coupon with valid fields and constraints for later use [FR-17].
3. As a customer, search for the new product, open details, add it to the cart, and proceed to checkout [FR-05] [FR-06] [FR-07].
4. Apply the newly created coupon and verify the discount and final amount reflect the configured rules [FR-09] [FR-08].
5. Complete checkout to confirm end-to-end data flow from admin configuration to user purchase [FR-08].

3. Comprehensive Tool Assessment

We surveyed 6 testing tools across the JavaScript and SaaS ecosystems to evaluate their capability in handling the asynchronous nature of a React SPA, strictly prioritizing Developer Experience (DX) within terminal-native (CLI) workflows.

3.1. Traditional Frameworks (Code-based)

Playwright

- **Characteristics:** Node/TypeScript-first automation that uses browser-native protocols with auto-waiting locators and strong SPA synchronization in headed or headless modes.
- **Pros:** High reliability for React rendering and network-driven UI, fast parallel execution with rich artifacts, and first-class CLI workflows that fit pure terminal and CI pipelines. Crucially, its native support for **Multiple Browser Contexts** allows seamless parallel testing.
- **Cons:** Smaller legacy ecosystem than Selenium and a stronger dependency on the Node toolchain; mobile testing is limited to emulation rather than real devices.
- **Best for:** React SPA end-to-end suites that demand speed, stability, and terminal-native execution.

Selenium 4

- **Characteristics:** W3C WebDriver-standard automation with multi-language bindings and Grid-based distributed execution.
- **Pros:** Broadest browser and tooling compatibility, mature enterprise integrations, solid interoperability with Appium for a broader web+mobile strategy, and the introduction of Relative Locators in version 4 for more flexible DOM querying.
- **Cons:** WebDriver latency can be slower for SPA-heavy flows, explicit waits require more engineering discipline, and infrastructure setup (Grid, drivers) adds overhead.
- **Best for:** Organizations needing cross-browser compliance at scale and leveraging existing Selenium expertise or heterogeneous language stacks.

3.2. AI-Augmented Tools (Low-code/Self-healing)

Testim AI

- **Characteristics:** SaaS-based low-code automation with AI-driven locator healing and cloud execution.
- **Pros:** Rapid authoring and maintenance reduction through self-healing locators, with a CLI runner that can plug into CI for React SPA regression suites.
- **Cons:** Vendor lock-in and cloud dependence can conflict with terminal-native, local-first workflows; debugging is less transparent than code-native frameworks.
- **Best for:** Teams prioritizing fast UI coverage and low maintenance cost over deep framework control.

mabl

- **Characteristics:** Cloud-native, low-code testing platform with AI-assisted maintenance and managed execution infrastructure.

- **Pros:** Quick ramp-up, built-in analytics and reporting, and consistent cross-browser runs without maintaining local infrastructure.
- **Cons:** Authoring is GUI-centric, limiting pure CLI workflows; customization for complex SPA edge cases can be constrained by platform abstractions.
- **Best for:** Product teams that need managed, low-ops web regression testing with minimal engineering investment.

3.3. Backup & Alternative Frameworks

Cypress

- **Characteristics:** JavaScript-based test runner that executes in the browser context with rich debugging and tight frontend integration.
- **Pros:** Excellent developer experience for React SPAs, fast local feedback loops, and reliable network stubbing with a strong CLI for headless runs.
- **Cons:** Multi-tab and multi-domain flows remain constrained, and tests are limited to the JavaScript ecosystem; real-device mobile coverage is out of scope.
- **Best for:** Frontend-heavy teams optimizing rapid iteration and tight SPA debugging in a terminal-centric workflow.

WebdriverIO

- **Characteristics:** Node.js WebDriver client with a flexible runner and plugin ecosystem that can target Selenium and Appium.
- **Pros:** Unifies web and mobile automation in one JavaScript stack, supports multiple test frameworks, and fits CLI-centric CI execution.
- **Cons:** WebDriver protocol overhead can slow dynamic SPA tests, and stability depends on disciplined waits and configuration.
- **Best for:** Teams wanting a single JS-based framework for both web and mobile automation with existing WebDriver infrastructure.

4. Comprehensive Comparison Matrix

The matrix below summarizes tool fit for a React SPA with a terminal-native workflow.

Tool	Licence Cost	Learning Curve	EShop Fit	AI Capability	Community
Playwright	Free/OSS (Apache 2.0)	Medium. CLI-first and highly coherent for terminal-native workflows.	High. Strong SPA sync and fast headless runs; no real-device mobile.	Low. No native AI, but easy to pair with LLM tooling.	Large. Rapidly growing ecosystem and active maintenance.

Tool	Licence Cost	Learning Curve	EShop Fit	AI Capability	Community
Selenium 4	Free/OSS (Apache 2.0)	Medium. Mature but heavier setup for drivers and Grid.	Medium. Broad browser support; SPA stability needs careful waits.	Low. No native AI; extensible via third-party tooling.	Massive. Longest-standing ecosystem and broad support.
Testim AI	Paid SaaS with limited free tier	Low. GUI-centric authoring reduces CLI purity.	Medium. Solid SPA coverage but limited mobile depth.	High. Self-healing locators and AI-driven maintenance.	Medium. Vendor-led community and support.
mabl	Paid SaaS with trial	Low. Platform-first workflow conflicts with pure CLI.	Medium. Reliable web coverage; mobile is out of scope.	High. Built-in AI-assisted maintenance and analytics.	Medium. Growing user base, platform-focused.
Cypress	Free/OSS (MIT) with paid dashboard	Low. Excellent local DX and fast feedback loop for JS teams.	High. Strong React SPA support; no real-device mobile.	Low. Limited native AI; extensible with plugins.	Large. Very active frontend community.
WebdriverIO	Free/OSS (MIT)	Medium. Flexible but initial configuration is heavy due to vast plugin choices.	High. Solid React SPA coverage; native Appium support perfectly fits the mobile requirement.	Low. No native AI, relies on ecosystem tooling.	Large. Established JS WebDriver community.

5. Final Recommendation & Rationale

We recommend adopting **Playwright augmented with GitHub Copilot** as the core automation stack for EShop.

- **Terminal-Native DX:** Unlike GUI-dependent tools such as Testim or mabl, Playwright is pure-CLI and code-first, integrating seamlessly into a native Ubuntu shell and terminal-based editing workflows without disrupting developer momentum.
- **Architectural Superiority for EShop:** Compared to Cypress, Playwright offers native Multiple Browser Contexts for parallel Admin/Customer flows (Journey 4) and robust auto-waiting that aligns with React's asynchronous DOM rendering.
- **Cost-Effective AI Integration:** Instead of paying for vendor-locked SaaS solely for self-healing, Playwright plus Copilot enables programmatic, resilient, `data-testid`-driven logic while keeping

the framework open-source and extensible.

6. AI Disclosure & Cross-Verification

- **AI Usage:** AI tools (Gemini and GitHub Copilot) were used to structure the report, synthesize theoretical concepts, and brainstorm locator resilience strategies.
- **Cross-Verification:** Architectural claims and tool comparisons were cross-referenced against official Playwright and Cypress documentation and independent technical blogs on AI self-healing mechanics.